HZZAnalysis

November 3, 2026

0.1 How to rediscover the Higgs boson yourself!

This notebook uses ATLAS Open Data http://opendata.atlas.cern 2025 beta release to show you the steps to rediscover the Higgs boson yourself!

ATLAS Open Data provides open access to proton-proton collision data at the LHC for educational purposes. ATLAS Open Data resources are ideal for high-school, undergraduate and postgraduate students.

Notebooks are web applications that allow you to create and share documents that can contain for example: 1. live code 2. visualisations 3. narrative text

0.1.1 What is the Higgs boson?

The Higgs boson is a fundamental particle predicted by the Standard Model. It is a manifestation of the Higgs field, which gives mass to the fundamental particles. However, it is incredibly hard to produce. At the LHC, a Higgs particle is produced about once every 10 billion collisions! This tiny fraction makes it very difficult to detect. Nevertheless, after years of data collection, the Higgs boson was finally discovered in 2012 by CMS and ATLAS experiments at CERN. In this tutorial, we shall be following their example.

0.1.2 Detecting the Higgs

This analysis loosely follows the paper on the discovery of the Higgs boson by ATLAS (mostly Section 4 and 4.1).

The Higgs boson can be produced in many different ways. In particle physics, we describe these production modes using Feynman diagrams. These diagrams allow us to visualise particle processes while also acting as powerful tools for calculations. See here for more information on Feynman diagrams.

There are four main production modes of the Higgs boson, and their respective Feynman diagrams: 1. Gluon-gluon fusion (top left) 2. Vector boson fusion (top right) 3. Vector boson bremsstrahlung (bottom left) 4. Top-antitop fusion (bottom right)

The Higgs has a very short lifetime, on the order of 10^{-22} s. It decays extremely quickly after production, so there is no hope of directly detecting the particle. Nevertheless, we can use the Standard Model to predict its decay products: photons, Z bosons, quarks, etc., all with different probabilities. These **decay channels** can be used to identify the Higgs boson. In this notebook, we'll be looking at one particular decay channel:

$$H \to ZZ^* \to \ell\ell\ell\ell$$

We refer to this as our desired **signal**. Ideally, we would search for collisions which yield four leptons as products and this would tell us that a Higgs boson is present. Unfortunately, in addition to our signal, there are many other **background** processes that lead to four reconstructed leptons in the final state. The main background is $ZZ^* \rightarrow \ell\ell\ell\ell$, where decay products have the same properties as those in the Higgs decay. This is known as an irreducible background.

We can get around this by accounting for the total invariant mass of the lepton products. We know through conservation of energy and momentum that the invariant mass of the products must be equal to the Higgs mass, while other background processes will have different invariant masses. Our last step would be to plot the invariant mass of each event and spot the peak in mass around 125 GeV, which corresponds to the mass of the Higgs boson.

We also have background contributions from $Z+$ jets and top-anti top processes, where additional charged leptons can arise either from semi-leptonic decays of heavy flavour or light flavour jets misidentified as leptons. We can also have contributions from processes with more than four leptons, but where one (or more) of the leptons are not reconstructed, and thus do not appear in our final selection. These backgrounds are difficult to remove completely.

For such processes, we will attempt to distinguish them from the Higgs decay using the properties of the leptons. Because the Higgs is a neutral particle with zero lepton number, the lepton products from its decay must sum to zero charge and zero lepton numbers. Thus, we can cut away all data with products that do not have these properties. These cuts increase the ratio of our signal to the reducible background.

Note: Z^*/W^* refer to a Z/W boson that is off its mass shell. This means that its mass is not fixed to the 91/80 GeV of a typical Z/W boson.

By the end of this notebook you will be able to: 1. Learn to process large data sets using cuts 2. Understand some general principles of a particle physics analysis 3. Discover the Higgs boson!

See here for more information on signals and backgrounds!

0.1.3 Running a Python notebook

A Python notebook consists of cell blocks, each containing lines of Python code. Each cell can be run independently of each other, yielding respective outputs below the cells. Conventionally, cells are run in order from top to bottom.

- To run the whole notebook, in the top menu click Cell → Run All.
- To propagate a change you've made to a piece of code, click Cell → Run All Below.
- You can also run a single code cell, by clicking Cell → Run Cells, or using the keyboard shortcut Shift+Enter.

For more information, refer to here.

0.2 ATLAS Open Data Initialisation

0.2.1 First time package installation on your computer (not needed on mybinder)

This first cell installs the required python packages. It only needs to be run the first time you open this notebook on your computer. If you close Jupyter and re-open on the same computer, you won't need to run this first cell again.

If this is opened on mybinder, you don't need to run this cell.

[ ]:
```
import sys
import os.path
!pip install atlasopenmagic
from atlasopenmagic import install_from_environment
install_from_environment()
```

We're going to import a number of packages to help us: * `numpy`: provides numerical calculations such as histogramming * `matplotlib`: common tool for making plots, figures, images, visualisations * `uproot`: processes `.root` files typically used in particle physics into data formats used in python * `awkward`: introduces `awkward` arrays, a format that generalizes `numpy` to nested data with possibly variable length lists * `vector`: to allow vectorized 4-momentum calculations

[2]:
```
import numpy as np # for numerical calculations such as histogramming
import matplotlib.pyplot as plt # for plotting
import matplotlib_inline # to edit the inline plot format
#matplotlib_inline.backend_inline.set_matplotlib_formats('pdf', 'svg') # to␣
 ↪make plots in pdf (vector) format
from matplotlib.ticker import AutoMinorLocator # for minor ticks
import uproot # for reading .root files
import awkward as ak # to represent nested data in columnar format
import vector # for 4-momentum calculations
import time # for printing time stamps
import requests # for file gathering, if needed
```

Unit definitions, as stored in the data files

[3]:
```
MeV = 0.001
GeV = 1.0
```

We will use the atlasopenmagic to access the open data directly from the ATLAS OpenData Portal so no need to download any samples. First we need to install the package

Import the module and load the release.

[4]:
```
import atlasopenmagic as atom
atom.available_releases()
atom.set_release('2025e-13tev-beta')
```

```
Available releases:
==========================================
2016e-8tev        2016 Open Data for education release of 8 TeV proton-proton
collisions (https://opendata.cern.ch/record/3860).
2020e-13tev       2020 Open Data for education release of 13 TeV proton-proton
collisions (https://cern.ch/2r7xt).
2024r-pp          2024 Open Data for research release for proton-proton
collisions (https://opendata.cern.record/80020).
2024r-hi          2024 Open Data for research release for heavy-ion collisions
(https://opendata.cern.ch/record/80035).
```

```
2025e-13tev-beta  2025 Open Data for education and outreach beta release for 13
TeV proton-proton collisions(https://opendata.cern.ch/record/93910).
2025r-evgen       2025 Open Data for research release for event generation
(https://opendata.cern.ch/record/160000).
Fetching and caching all metadata for release: 2025e-13tev-beta…
Successfully cached 374 datasets.
Active release: 2025e-13tev-beta. (Datasets path: REMOTE)
```

0.3 Example 1: Reading data

We would like to read some of the data from the open dataset.

[5]:
```
lumi = 36.6 # fb-1 # data size of the full release
fraction = 1.0 # reduce this is if you want the code to run quicker
```

[6]:
```
# Select the skim to use for the analysis
skim = "exactly4lep"
```

For convenient naming and identification purposes, we define a dictionary which stores all the important names of the samples we want to pull from the database.

[ ]:
```
defs = {
    r'Data':{'dids':['data']},
    r'Background $Z,t\bar{t},t\bar{t}+V,VVV$':{'dids': [410470,410155,410218,
                                                        410219,412043,364243,
                                                        364242,364246,364248,
                                                        700320,700321,700322,
                                                        700323,700324,700325],␣
 ↪'color': "#6b59d3" }, # purple
    r'Background $ZZ^{*}$':     {'dids': [700600],'color': "#ff0000" },# red
    r'Signal ($m_H$ = 125 GeV)':  {'dids': [345060, 346228, 346310, 346311,␣
 ↪346312,
                                           346340, 346341, 346342],'color':␣
 ↪"#00cdff" },# light blue
}

samples   = atom.build_dataset(defs, skim=skim, protocol='https', cache=True)
```

The key named `data` refers to the event information collected from real experiments, while the `Background` and `Signal` keys refer to Monte-Carlo (MC) simulations of the ATLAS experiments. Both real data and MC data will then be analysed and compared together to discover the Higgs!

Let's try accessing `data15_periodD` in the CERN database URL as an example.

[9]:
```
# We shall use the first entry in 'list', 'data15_periodD'
data15_periodD = samples['Data']['list'][0]
print(f"{data15_periodD = }")
```

```
data15_periodD = 'simplecache::https://opendata.cern.ch/eos/opendata/atlas/rucio
/opendata/ODEO_FEB2025_v0_exactly4lep_data15_periodD.exactly4lep.root'
```

Next, we shall try opening the `data15_periodD` file to see what is inside. In the file (called a `tree`), there are 39 entries, one for each event. In each event, a dictionary stores the all relevant information as keys, such as the event number (`eventNumber`), lepton transverse momentum (`lep_pt`), etc. Details on the variables in the dictionary can be viewed here.

More information on trees can be viewed here.

[ ]:
```
# Accessing the file from the online database (":analysis" opens the tree in a␣
 ↪desired manner)
tree = uproot.open(data15_periodD + ":analysis")

# There are 39 entries in the tree
print(tree.num_entries)

# We can view all the information stored in the tree using the .keys() method.
print(tree.keys())

# We can also view the entire tree using the .arrays() method
# This generates a 39-entry list of dictionaries
print(tree.arrays())
```

Perhaps we'd like to see the lepton energies. We can access this from our tree using the key `lep_e`. Also, from this point on we shall be manipulating our tree arrays using the `awkward` library. We can use `library="ak"` in the argument of the `.arrays()` method to use this library. If you ever see `library="ak"` in the code, it means that the array is output as an `awkward` array.

[ ]:
```
tree["lep_e"].arrays(library="ak")
```

In our analysis, not all the information in the tree is important. We can store the important variables in a list and retrieve them from the tree later on. As it turns out, we will need the following set of variables:

[12]:
```
# Define what variables are important to our analysis
variables =␣
 ↪['lep_pt','lep_eta','lep_phi','lep_e','lep_charge','lep_type','trigE','trigM','lep_isTrigMa
           'lep_isLooseID','lep_isMediumID','lep_isLooseIso','lep_type']

# To see all the data for our given variables
# for data in tree.iterate(variables, library="ak"):
#     print(data)
```

Now that we understand how to access the information in the `data15_periodD` tree, we can begin analysis. As mentioned in the introduction, there are two key steps to be completed for each event entry: 1. **Cuts** - we need to account for lepton selection rules in the event. In the paper, it is stated that we must "[select] two pairs of isolated leptons, each of which is comprised of two leptons with the **same flavour** and **opposite charge**". The datasets used in this notebook have already been filtered to include at least 4 leptons per event. We need to filter the data such that in each event, there are pairs of leptons of the **same lepton type** (`lep_type`) and summing to **zero lepton charge** (`lep_charge`).

2. **Mass calculation** - the data to be plotted is the 4-lepton invariant mass, which can be found using the equation:

$$m_{4l} = \sqrt{E_{\text{tot}}^2 - \mathbf{p}_{\text{tot}} \cdot \mathbf{p}_{\text{tot}}}$$

in units where $c = 1$. E_{tot} is the total energy and $\mathbf{p}_{\text{tot}}$ is the total momentum. This calculation is performed using the vector array method `.M` on the sum of lepton 4-momenta (`lep_pt`,`lep_eta`,`lep_phi`,`lep_E`).

From this, we can see why we chose those six important variables earlier. The physical reasoning for why we perform these steps is encapsulated in the idea of **conservation laws**. You may read more here.

Let's try to perform this two-step analysis for one event in `data15_periodD`.

Define function to get data from files.

The datasets used in this notebook have already been filtered to include at least 4 leptons per event, so that processing is quicker.

[13]:
```
# This selects the first entry of the tree
entry = tree.arrays(library="ak")[:1]

# Cut lepton type (electron type is 11,  muon type is 13)
lep_type = entry['lep_type']
sum_lep_type = lep_type[:, 0] + lep_type[:, 1] + lep_type[:, 2] + lep_type[:, 3]
lep_type_cut_bool = (sum_lep_type != 44) & (sum_lep_type != 48) & (sum_lep_type␣
 ↪!= 52)
print(f"Cut for lepton type? {lep_type_cut_bool}") # True means we should␣
 ↪remove this entry (lepton type does not match)

# Cut lepton charge
# first lepton in each event is [:, 0], 2nd lepton is [:, 1] etc
lep_charge = entry['lep_charge']
sum_lep_charge = lep_charge[:, 0] + lep_charge[:, 1] + lep_charge[:, 2] +␣
 ↪lep_charge[:, 3] != 0
print(f"Cut for lepton charge? {sum_lep_charge}") # True means we should remove␣
 ↪this entry (sum of lepton charges is not equal to 0)

# Calculate invariant mass of the 4-lepton state
# [:, i] selects the i-th lepton in each event
p4 = vector.zip({"pt": entry['lep_pt'], "eta": entry['lep_eta'], "phi":␣
 ↪entry['lep_phi'], "E": entry['lep_e']})
invariant_mass = (p4[:, 0] + p4[:, 1] + p4[:, 2] + p4[:, 3]).M # .M calculates␣
 ↪the invariant mass
print(f"Invariant mass: {invariant_mass}")
```

```
Cut for lepton type? [True]
Cut for lepton charge? [False]
Invariant mass: [5.6]
```

Based on our analysis, this entry should be removed because the lepton type does not match our requirements. We can turn these checks and calculations into a set of functions.

[14]:
```
# Cut lepton type (electron type is 11,  muon type is 13)
def cut_lep_type(lep_type):
    sum_lep_type = lep_type[:, 0] + lep_type[:, 1] + lep_type[:, 2] + lep_type[:
 ↪, 3]
    lep_type_cut_bool = (sum_lep_type != 44) & (sum_lep_type != 48) &␣
 ↪(sum_lep_type != 52)
    return lep_type_cut_bool # True means we should remove this entry (lepton␣
 ↪type does not match)

# Cut lepton charge
def cut_lep_charge(lep_charge):
    # first lepton in each event is [:, 0], 2nd lepton is [:, 1] etc
    sum_lep_charge = lep_charge[:, 0] + lep_charge[:, 1] + lep_charge[:, 2] +␣
 ↪lep_charge[:, 3] != 0
    return sum_lep_charge # True means we should remove this entry (sum of␣
 ↪lepton charges is not equal to 0)

# Calculate invariant mass of the 4-lepton state
# [:, i] selects the i-th lepton in each event
def calc_mass(lep_pt, lep_eta, lep_phi, lep_e):
    p4 = vector.zip({"pt": lep_pt, "eta": lep_eta, "phi": lep_phi, "E": lep_e})
    invariant_mass = (p4[:, 0] + p4[:, 1] + p4[:, 2] + p4[:, 3]).M # .M␣
 ↪calculates the invariant mass
    return invariant_mass


def cut_trig_match(lep_trigmatch):
    trigmatch = lep_trigmatch
    cut1 = ak.sum(trigmatch, axis=1) >= 1
    return cut1

def cut_trig(trigE,trigM):
    return trigE | trigM


def ID_iso_cut(IDel,IDmu,isoel,isomu,pid):
    thispid = pid
    return (ak.sum(((thispid == 13) & IDmu & isomu) | ((thispid == 11) & IDel &␣
 ↪isoel), axis=1) == 4)
```

You may verify on your own that these functions give the same outputs as the previous code block. Now, we shall apply these functions over the entire data tree using a `for` loop.

[15]:
```
# Define empty list to hold all data for this sample
sample_data = []

# Perform the cuts for each data entry in the tree
for data in tree.iterate(variables, library="ak"): # the data will be in the␣
 ↪form of an awkward array
    # We can use data[~boolean] to remove entries from the data set
    lep_type = data['lep_type']
    data = data[~cut_lep_type(lep_type)]
    lep_charge = data['lep_charge']
    data = data[~cut_lep_charge(lep_charge)]

    data['mass'] = calc_mass(data['lep_pt'], data['lep_eta'], data['lep_phi'],␣
 ↪data['lep_e'])

    # Append data to the whole sample data list
    sample_data.append(data)

# turns sample_data back into an awkward array
data_A = ak.concatenate(sample_data)
```

We can now plot the data using Matplotlib. The data will be turned into a histogram, with bins of width 2.5 GeV. Note that much of the code written here is meant for the aesthetics of the plot.

[16]:
```
# x-axis range of the plot
xmin = 80 * GeV
xmax = 250 * GeV

# Histogram bin setup
step_size = 2.5 * GeV
bin_edges = np.arange(start=xmin, # The interval includes this value
                      stop=xmax+step_size, # The interval doesn't include this␣
 ↪value
                      step=step_size ) # Spacing between values
bin_centres = np.arange(start=xmin+step_size/2, # The interval includes this␣
 ↪value
                         stop=xmax+step_size/2, # The interval doesn't include␣
 ↪this value
                         step=step_size ) # Spacing between values

# Creating histogram from data
data_x,_ = np.histogram(ak.to_numpy(data_A['mass']),
                         bins=bin_edges ) # histogram the data
data_x_errors = np.sqrt( data_x ) # statistical error on the data

# *************
# Main plot
```

```
# *************
main_axes = plt.gca() # get current axes

# plot the data points
main_axes.errorbar(x=bin_centres, y=data_x, yerr=data_x_errors,
                   fmt='ko', # 'k' means black and 'o' is for circles
                   label='Data')

# set the x-limit of the main axes
main_axes.set_xlim( left=xmin, right=xmax )

# separation of x axis minor ticks
main_axes.xaxis.set_minor_locator( AutoMinorLocator() )

# set the axis tick parameters for the main axes
main_axes.tick_params(which='both', # ticks on both x and y axes
                      direction='in', # Put ticks inside and outside the axes
                      top=True, # draw ticks on the top axis
                      right=True ) # draw ticks on right axis

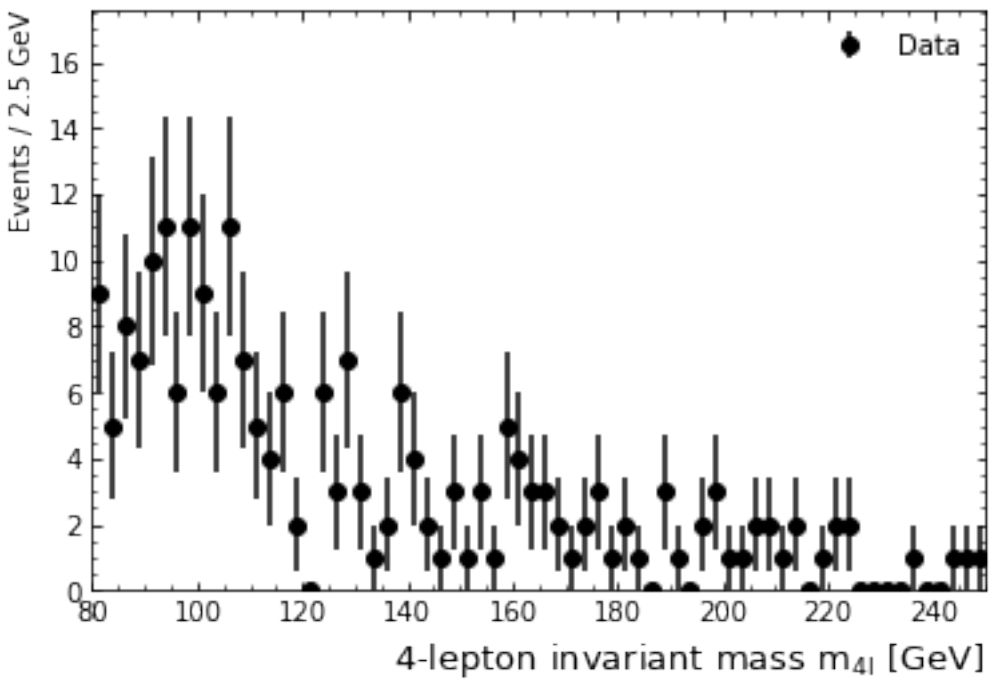
# x-axis label
main_axes.set_xlabel(r'4-lepton invariant mass $\mathrm{m_{4l}}$ [GeV]',
                   fontsize=13, x=1, horizontalalignment='right' )

# write y-axis label for main axes
main_axes.set_ylabel('Events / '+str(step_size)+' GeV',
                      y=1, horizontalalignment='right')

# set y-axis limits for main axes
main_axes.set_ylim( bottom=0, top=np.amax(data_x)*1.6 )

# add minor ticks on y-axis for main axes
main_axes.yaxis.set_minor_locator( AutoMinorLocator() )

# draw the legend
main_axes.legend( frameon=False ); # no box around the legend
```



Great, we managed to plot `data15_periodD`! Now, we have not discussed how to deal with the Monte-Carlo simulation data, or even what they are for. Let us explain.

0.4 Example 2: Reading Monte-Carlo data

Using the Standard Model, we can do a set of randomised simulations to produce a set of theoretical data points to compare to our ATLAS data. These are known as Monte-Carlo(MC) simulations. But there is one important change to be made to the MC data before we can compare them with our ATLAS data:

- **Weights** - The MC data was simulated with *ideal* or *controlled* circumstances. However, the real ATLAS detector has some inefficiencies, which we can account for by attributing an appropriate weight to each data point. The weight of a data point affects how it contributes to the histogram count for its bin.

Let's open an MC file.

[17]:
```
# We open an MC data file with simulated ttbar events (data set ID 410470)
value = samples[r'Background $Z,t\bar{t},t\bar{t}+V,VVV$']["list"][0]
print(value)

# This is now appended to our file path to retrieve the root file
background_ttbar_path = value

# Accessing the file from the online database
tree = uproot.open(background_ttbar_path + ":analysis")
```

```
simplecache::https://opendata.cern.ch/eos/opendata/atlas/rucio/opendata/ODEO_FEB
2025_v0_exactly4lep_mc_410470.PhPy8EG_A14_ttbar_hdamp258p75_nonallhad.exactly4le
p.root
```

These are the weights which are important to our analysis: - `scaleFactor_PILEUP` - scale factor for pileup reweighting - `scaleFactor_ELE` - scale factor for electron efficiency - `scaleFactor_MUON`- scale factor for muon efficiency - `scaleFactor_LepTRIGGER` - scale factor for lepton triggers

Scale factors are generally related to estimates of the efficiencies and resolutions of detectors.

[18]:
```
weight_variables = ["filteff","kfac","xsec","mcWeight","ScaleFactor_PILEUP",␣
↪"ScaleFactor_ELE", "ScaleFactor_MUON", "ScaleFactor_LepTRIGGER"]

# For example, see below for the weights corresponding to muon rejection
tree["ScaleFactor_MUON"].arrays(library = "ak")
print(weight_variables)
```

```
['filteff', 'kfac', 'xsec', 'mcWeight', 'ScaleFactor_PILEUP', 'ScaleFactor_ELE',
'ScaleFactor_MUON', 'ScaleFactor_LepTRIGGER']
```

Additionally, there is a *cross section weight* w_σ associated with each MC file. This weight is meant to normalise the entire Monte-Carlo distribution based on the number of events in the data. This is its definition:

$$w_\sigma = \frac{\int L\mathrm{d}t \ \sigma \ \eta \ k \ w}{\sum_i w_i}$$

where * $\int L\mathrm{d}t$ is the integrated luminosity in inverse femtobarns (`lumi`), * σ is the cross section in picobarn (hence the factor 1000 in the formula below) (`event["xsec"]`), * η is the filter efficiency of the MC generator (`event["filteff"]`), * k is the k-factor, a multiplicative correction factor used to account for higher-order effects in theoretical calculations of the cross section (`event["kfac"]`). * w are weights added to a given simulated event by the event generator. These can be set to 1 for many sampels and depends on the process being simulated and the generator used to simulate the events (`event["mcWeight"]`). * In datasets where events are assigned weights, the $\sum_i w_i$ value represents the cumulative total of these weights across all events in the dataset (`event["sum_of_weights"]`). If no weights are assigned to the events this (i.e. the weights are all set to 1) $\sum_i w_i$ simply gives the total number of simulated events. Note that the sum of weights is calculated before any selection has been applied and thus is typically much larger than what you get if you sum up the weights of the events in the ntuple.

When the integrated luminosity is multiplied by the *cross section weight*, w_σ, of a certain process, it gives a measure of the total number of expected events during a period of data taking for the given process.

For `data_D`, the integrated luminosity has a value of $0.105\,\mathrm{fb}^{-1}$.

For more on cross sections and luminosities, see this cheatsheet.

Now, with all the weights we've defined, we will calculate a total weight for an event in the $t\bar{t}$ sample scaled to the integrated luminosity of period D in data15. The total weight is the collective product of all the weights:

[19]:
```
# luminosity of data_D
lumi = 0.105 #Luminosity of periodD in data 15 is 0.105 fb-1

# Let's use the first event of our tree
event = tree.arrays()[0]

# Multiply all the important weights together
total_weight = lumi * 1000 / event["sum_of_weights"]
for variable in weight_variables:
    total_weight = total_weight * event[variable]
print(f"{total_weight = }")
```

```
total_weight = np.float64(0.0017336209613845047)
```

This calculation means that in our final histogram, this event will be represented with 0.00173 of a single count in the bin. We can encapsulate these calculations in a single function `calc_weight`.

[20]:
```
def calc_weight(weight_variables, events):
    total_weight = lumi * 1000 / events["sum_of_weights"]
    for variable in weight_variables:
        total_weight = total_weight * abs(events[variable])
    return total_weight

# Verify that we get the same answer
print(calc_weight(weight_variables, event))
```

```
0.0017336209613845047
```

Now, we can apply the cuts as before to plot the MC data. The code is the same as before, but we make sure to add in `weight_variables` to our `tree.iterate()`, and we store the weights in each event using a new dictionary key.

[21]:
```
sample_data = []

# Perform the cuts for each data entry in the tree
for data in tree.iterate(variables + weight_variables+['sum_of_weights'],␣
 ↪library="ak"):
    # Cuts
    lep_type = data['lep_type']
    data = data[~cut_lep_type(lep_type)]
    lep_charge = data['lep_charge']
    data = data[~cut_lep_charge(lep_charge)]

    # Invariant Mass
    data['mass'] = calc_mass(data['lep_pt'], data['lep_eta'], data['lep_phi'],␣
 ↪data['lep_e'])

    # Store Monte Carlo weights in the data
    data['totalWeight'] = calc_weight(weight_variables, data)
```

```
    # data['totalWeight'] = calc_weight(data)

    # Append data to the whole sample data list
    sample_data.append(data)

# turns sample_data back into an awkward array
background_ttbar = ak.concatenate(sample_data)
```

[22]:
```
step_size = 2.5 * GeV

mc_x = ak.to_numpy(background_ttbar["mass"]) # define list to hold the Monte␣
 ↪Carlo histogram entries
mc_weights = ak.to_numpy(background_ttbar["totalWeight"]) # define list to hold␣
 ↪the Monte Carlo weights
mc_colors = samples[r'Background $Z,t\bar{t},t\bar{t}+V,VVV$']['color'] #␣
 ↪define list to hold the colors of the Monte Carlo bars
mc_labels = r'Background $Z,t\bar{t},t\bar{t}+V,VVV$' # define list to hold the␣
 ↪legend labels of the Monte Carlo bars

# *************
# Main plot
# *************
main_axes = plt.gca() # get current axes

# plot the data points
# main_axes.errorbar(x=bin_centres, y=data_x, yerr=data_x_errors,
#                    fmt='ko', # 'k' means black and 'o' is for circles
#                    label='Data')

# plot the Monte Carlo bars
mc_heights = main_axes.hist(mc_x, bins=bin_edges,
                            weights=mc_weights, stacked=True,
                            color=mc_colors, label=mc_labels )

mc_x_tot = mc_heights[0] # stacked background MC y-axis value

# calculate MC statistical uncertainty: sqrt(sum w^2)
mc_x_err = np.sqrt(np.histogram(np.hstack(mc_x), bins=bin_edges, weights=np.
 ↪hstack(mc_weights)**2)[0])

# plot the statistical uncertainty
main_axes.bar(bin_centres, # x
              2*mc_x_err, # heights
              alpha=0.5, # half transparency
              bottom=mc_x_tot-mc_x_err, color='none',
              hatch="////", width=step_size, label='Stat. Unc.' )
```

```
# set the x-limit of the main axes
main_axes.set_xlim( left=xmin, right=xmax )

# separation of x axis minor ticks
main_axes.xaxis.set_minor_locator( AutoMinorLocator() )

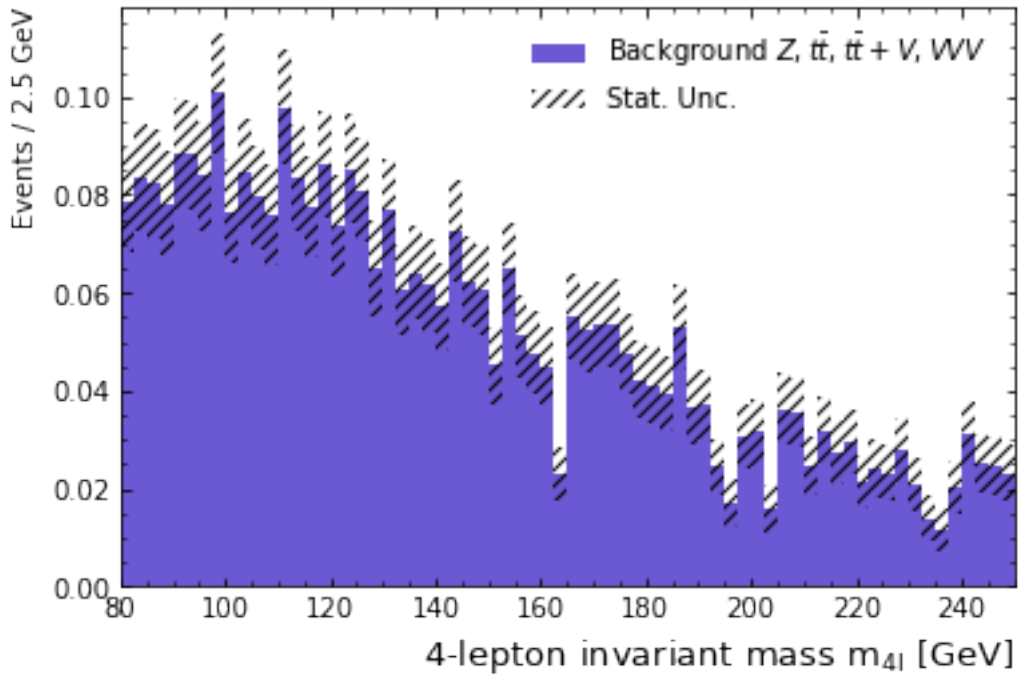
# set the axis tick parameters for the main axes
main_axes.tick_params(which='both', # ticks on both x and y axes
                      direction='in', # Put ticks inside and outside the axes
                      top=True, # draw ticks on the top axis
                      right=True ) # draw ticks on right axis

# x-axis label
main_axes.set_xlabel(r'4-lepton invariant mass $\mathrm{m_{4l}}$ [GeV]',
                     fontsize=13, x=1, horizontalalignment='right' )

# write y-axis label for main axes
main_axes.set_ylabel('Events / '+str(step_size)+' GeV',
                      y=1, horizontalalignment='right')

# add minor ticks on y-axis for main axes
main_axes.yaxis.set_minor_locator( AutoMinorLocator() )

# draw the legend
main_axes.legend( frameon=False ); # no box around the legend
```



0.5 Final Analysis

Now that we understand all the steps of our analysis, all that's left is to import the entire ATLAS data and implement it. The `samples` dictionary will be useful for this.

We will loop over all values in the `samples` dictionary. Depending on whether it is a data sample or MC sample, `fileString` will change, which opens the correct file on the open data folder. As before, the cuts, mass calculations and MC weight calculations will be performed for each sample value, and then stored in the array. The data will all be concatenated into `all_data` for plotting.

[ ]:
```
# Set luminosity to 36.6 fb-1, data size of the full release
lumi = 36.6

# Controls the fraction of all events analysed
fraction = 1.0 # reduce this is if you want quicker runtime (implemented in the␣
 ↪loop over the tree)

# Define empty dictionary to hold awkward arrays
all_data = {}

# Loop over samples
for s in samples:

    # Print which sample is being processed
    print('Processing '+s+' samples')

    # Define empty list to hold data
    frames = []

    # Loop over each file
    for val in samples[s]['list']:
        if s == 'data':
            prefix = "Data/" # Data prefix
        else: # MC prefix
            prefix = "MC/mc_"
        fileString = val

        # start the clock
        start = time.time()
        print("\t"+val+":")

        # Open file
        tree = uproot.open(fileString + ":analysis")

        sample_data = []
```

```
        # Loop over data in the tree
        for data in tree.iterate(variables + weight_variables +␣
↪["sum_of_weights", "lep_n"],
                                 library="ak",
                                 entry_stop=tree.num_entries*fraction):#, #␣
↪process up to numevents*fraction
                                #  step_size = 10000000):

            # Number of events in this batch
            nIn = len(data)

            data = data[cut_trig(data.trigE, data.trigM)]
            data = data[cut_trig_match(data.lep_isTrigMatched)]

            # Record transverse momenta (see bonus activity for explanation)
            data['leading_lep_pt'] = data['lep_pt'][:,0]
            data['sub_leading_lep_pt'] = data['lep_pt'][:,1]
            data['third_leading_lep_pt'] = data['lep_pt'][:,2]
            data['last_lep_pt'] = data['lep_pt'][:,3]

            # Cuts on transverse momentum
            data = data[data['leading_lep_pt'] > 20]
            data = data[data['sub_leading_lep_pt'] > 15]
            data = data[data['third_leading_lep_pt'] > 10]

            data = data[ID_iso_cut(data.lep_isLooseID,
                                   data.lep_isMediumID,
                                   data.lep_isLooseIso,
                                   data.lep_isLooseIso,
                                   data.lep_type)]

            # Number Cuts
            #data = data[data['lep_n'] == 4]

            # Lepton cuts

            lep_type = data['lep_type']
            data = data[~cut_lep_type(lep_type)]
            lep_charge = data['lep_charge']
            data = data[~cut_lep_charge(lep_charge)]

            # Invariant Mass
            data['mass'] = calc_mass(data['lep_pt'], data['lep_eta'],␣
↪data['lep_phi'], data['lep_e'])

            # Store Monte Carlo weights in the data
            if 'data' not in s: # Only calculates weights if the data is MC
```

```
                data['totalWeight'] = calc_weight(weight_variables, data)
                # data['totalWeight'] = calc_weight(data)

            # Append data to the whole sample data list
            sample_data.append(data)

            if not 'data' in val:
                nOut = sum(data['totalWeight']) # sum of weights passing cuts␣
 ↪in this batch
            else:
                nOut = len(data)

            elapsed = time.time() - start # time taken to process
            print("\t\t nIn: "+str(nIn)+",\t nOut: \t"+str(nOut)+"\t in␣
 ↪"+str(round(elapsed,1))+"s") # events before and after

        frames.append(ak.concatenate(sample_data))

    all_data[s] = ak.concatenate(frames) # dictionary entry is concatenated␣
 ↪awkward arrays
```

[24]:
```
data_x,_ = np.histogram(ak.to_numpy(all_data['Data']['mass']),
                        bins=bin_edges ) # histogram the data
data_x_errors = np.sqrt( data_x ) # statistical error on the data

signal_x = ak.to_numpy(all_data[r'Signal ($m_H$ = 125 GeV)']['mass']) #␣
 ↪histogram the signal
signal_weights = ak.to_numpy(all_data[r'Signal ($m_H$ = 125 GeV)'].totalWeight)␣
 ↪# get the weights of the signal events
signal_color = samples[r'Signal ($m_H$ = 125 GeV)']['color'] # get the colour␣
 ↪for the signal bar

mc_x = [] # define list to hold the Monte Carlo histogram entries
mc_weights = [] # define list to hold the Monte Carlo weights
mc_colors = [] # define list to hold the colors of the Monte Carlo bars
mc_labels = [] # define list to hold the legend labels of the Monte Carlo bars

for s in samples: # loop over samples
    if s not in ['Data', r'Signal ($m_H$ = 125 GeV)']: # if not data nor signal
        mc_x.append( ak.to_numpy(all_data[s]['mass']) ) # append to the list of␣
 ↪Monte Carlo histogram entries
        mc_weights.append( ak.to_numpy(all_data[s].totalWeight) ) # append to␣
 ↪the list of Monte Carlo weights
        mc_colors.append( samples[s]['color'] ) # append to the list of Monte␣
 ↪Carlo bar colors
        mc_labels.append( s ) # append to the list of Monte Carlo legend labels
```

```
# *************
# Main plot
# *************
fig, main_axes = plt.subplots(figsize=(12, 8))

# plot the data points
main_axes.errorbar(x=bin_centres, y=data_x, yerr=data_x_errors,
                   fmt='ko', # 'k' means black and 'o' is for circles
                   label='Data')

# plot the Monte Carlo bars
mc_heights = main_axes.hist(mc_x, bins=bin_edges,
                            weights=mc_weights, stacked=True,
                            color=mc_colors, label=mc_labels )

mc_x_tot = mc_heights[0][-1] # stacked background MC y-axis value

# calculate MC statistical uncertainty: sqrt(sum w^2)
mc_x_err = np.sqrt(np.histogram(np.hstack(mc_x), bins=bin_edges, weights=np.
 ↪hstack(mc_weights)**2)[0])

# plot the signal bar
signal_heights = main_axes.hist(signal_x, bins=bin_edges, bottom=mc_x_tot,
                weights=signal_weights, color=signal_color,
                label=r'Signal ($m_H$ = 125 GeV)')

# plot the statistical uncertainty
main_axes.bar(bin_centres, # x
                2*mc_x_err, # heights
                alpha=0.5, # half transparency
                bottom=mc_x_tot-mc_x_err, color='none',
                hatch="////", width=step_size, label='Stat. Unc.' )

# set the x-limit of the main axes
main_axes.set_xlim( left=xmin, right=xmax )

# separation of x axis minor ticks
main_axes.xaxis.set_minor_locator( AutoMinorLocator() )

# set the axis tick parameters for the main axes
main_axes.tick_params(which='both', # ticks on both x and y axes
                      direction='in', # Put ticks inside and outside the axes
                      top=True, # draw ticks on the top axis
                      right=True ) # draw ticks on right axis

# x-axis label
```

```
main_axes.set_xlabel(r'4-lepton invariant mass $\mathrm{m_{4l}}$ [GeV]',
                     fontsize=13, x=1, horizontalalignment='right' )

# write y-axis label for main axes
main_axes.set_ylabel('Events / '+str(step_size)+' GeV',
                         y=1, horizontalalignment='right')

# set y-axis limits for main axes
main_axes.set_ylim( bottom=0, top=np.amax(data_x)*2.0 )

# add minor ticks on y-axis for main axes
main_axes.yaxis.set_minor_locator( AutoMinorLocator() )

# Add text 'ATLAS Open Data' on plot
plt.text(0.1, # x
            0.93, # y
            'ATLAS Open Data', # text
            transform=main_axes.transAxes, # coordinate system used is that of␣
 ↪main_axes
            fontsize=16 )

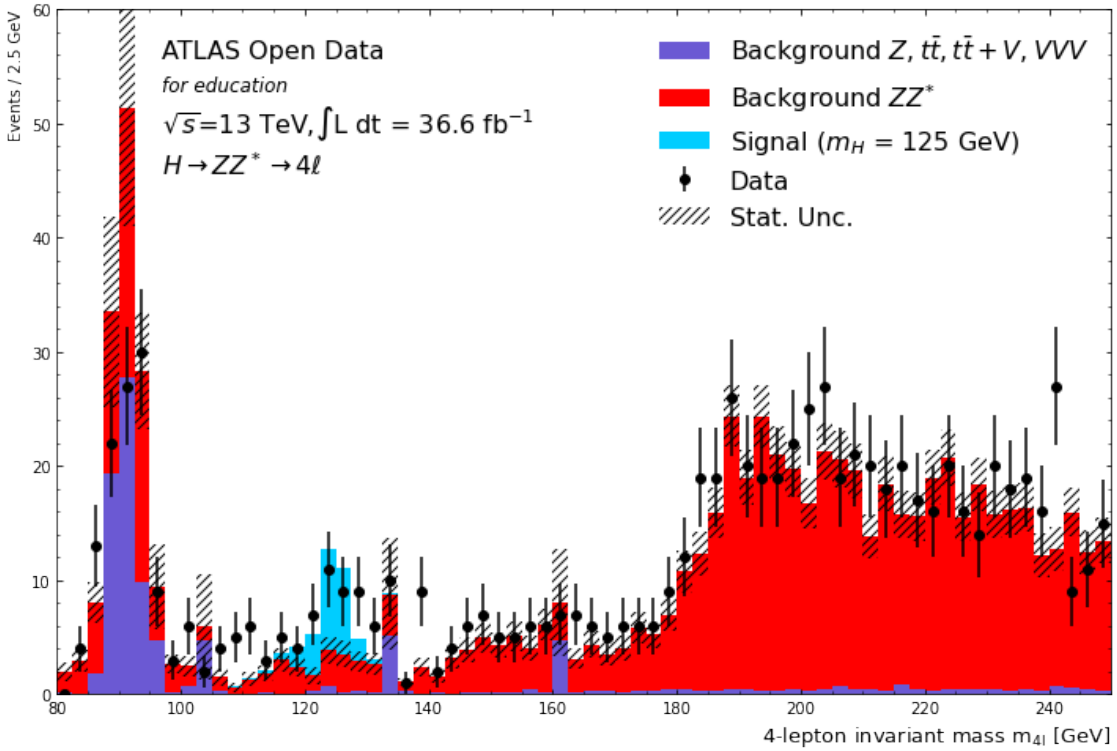
# Add text 'for education' on plot
plt.text(0.1, # x
            0.88, # y
            'for education', # text
            transform=main_axes.transAxes, # coordinate system used is that of␣
 ↪main_axes
            style='italic',
            fontsize=12 )

# Add energy and luminosity
lumi_used = str(lumi*fraction) # luminosity to write on the plot
plt.text(0.1, # x
            0.82, # y
            r'$\sqrt{s}$=13 TeV,$\int$L dt = '+lumi_used+' fb$^{-1}$', # text
            transform=main_axes.transAxes,fontsize=16 ) # coordinate system␣
 ↪used is that of main_axes

# Add a label for the analysis carried out
plt.text(0.1, # x
            0.76, # y
            r'$H \rightarrow ZZ^* \rightarrow 4\ell$', # text
            transform=main_axes.transAxes,fontsize=16 ) # coordinate system␣
 ↪used is that of main_axes

# draw the legend
```

```
my_legend = main_axes.legend( frameon=False, fontsize=16 ) # no box around the␣
↪legend
```



0.5.1 Signal Significance

We can do some analysis to study how significant the signal is compared to the background. One method is to check a quantity known as the signal significance S, which is defined by

$$S = \frac{N_{\text{sig}}}{\sqrt{N_{\text{bg}}}}$$

where $ N_sig $ and N_{bg} are the number of signal and background points respectively. A larger S represents a better signal-to-background ratio, and a more significant signal peak. To calculate N_{sig}, we can look at the plot and sum over the number of events of our Monte-Carlo signal. The signal range roughly corresponds to the bins from 115 GeV to 130 GeV. N_{bg} then corresponds to the number of background events in those same bins.

[25]:
```
# Signal stacked height
signal_tot = signal_heights[0] + mc_x_tot

# Peak of signal
print(signal_tot[18])

# Neighbouring bins
```

```python
print(signal_tot[17:20])

# Signal and background events
N_sig = signal_tot[17:20].sum()
N_bg = mc_x_tot[17:20].sum()

# Signal significance calculation
signal_significance = N_sig/np.sqrt(N_bg + 0.3 * N_bg**2) # EXPLAIN THE 0.3
print(f"\nResults:\n{N_sig = }\n{N_bg = }\n{signal_significance = }\n")
```

```
11.094884981667025
[12.69896722 11.09488498  4.93528783]

Results:
N_sig = np.float64(28.729140024845144)
N_bg = np.float64(10.300928352079978)
signal_significance = np.float64(4.425965281174207)
```

We welcome your feedback on this notebook or any of our other materials! Please fill out this survey to let us know how we're doing, and you can enter a raffle to win some ATLAS merchandise!